

Pi Web Android Native Client: v1.1

Deep Implementation & Full Protocol Alignment

Executive Summary & Overview

The Pi Web Android Native Client is a mobile-first application designed to provide a high-performance, native interface for the Pi Web backend ecosystem. Inspired by the architectural patterns of oc-remote, this client bridges mobile devices with remote LLM services, ensuring low-latency communication and a seamless user experience. By leveraging Kotlin, Jetpack Compose, and modern asynchronous networking, the project delivers a robust environment for real-time prompt execution and session management.

Architecture & Protocol Audit

Audit against pi-web backend ensures full API coverage for real-time state management and persistent terminal sessions.

REST API Endpoints

RESTful communication handles persistence and session discovery:

Endpoint Group	Actions	Protocol Description
Sessions & Flow	GET/POST	Discovery, Create, Fork, Abort, and Compact actions for session lifecycle.
Tool Calls	WS/POST	Execution and result broadcasting for external tool interactions.
Permissions	POST	respondPermission for handling user-gated backend requests.

Endpoint Group	Actions	Protocol Description
Terminals	WS	Interactive terminal sessions for remote CLI execution.
Metrics	Stream	Real-time Token and Cost metric tracking per generation task.

Upgraded Network Engine

For real-time token generation and interactive tool calls, the client maintains a persistent WebSocket connection:

- **Token Generation:** Individual text segments are pushed as `token` events for immediate UI rendering.
- **Tool Calls:** Lifecycle events for external tool execution (e.g., searching, calculation) are broadcast to notify the UI of background processes.
- **Lifecycle Events:** Includes `stream_start`, `stream_end`, and `error` states to manage UI loaders and state transitions.

Project Repository Structure

The project follows a standard Gradle-based Android structure, adhering to the Clean Architecture principles within an MVVM pattern.

- `.github/workflows/`: Contains CI/CD automation scripts.
- `app/build.gradle.kts`: Module-level configuration and dependency management.
- `app/src/main/`:
 - `AndroidManifest.xml`: Application permissions and entry points.
 - `java/com/piweb/client/`:
 - `data/`: Data models and networking logic (`Models.kt`, `PiWebClient.kt`).
 - `ui/`: Jetpack Compose screens and themes (`MainActivity.kt`, `Screens.kt`).
 - `viewmodel/`: State and flow management (`MainViewModel.kt`).

Deep Implementation Details (v1.1)

Rich Data Models

File: Models.kt

Kotlin

```
@Serializable data class SessionInfo(val id: String, val status: SessionStatus)
@Serializable data class ToolCall(val callId: String, val toolName: String)
@Serializable data class TokenUsage(val count: Int, val costMetrics: Double)
@Serializable data class PermissionRequest(val requestId: String, val scope: String)
```

Build Configuration

****File:**** `build.gradle.kts` (App Module)

```kotlin

plugins {

id("com.android.application")

id("org.jetbrains.kotlin.android")

}

android {

namespace = "com.piweb.client"

compileSdk = 34

defaultConfig {

applicationId = "com.piweb.client"

```
 minSdk = 26

 targetSdk = 34

 versionCode = 1

 versionName = "1.0"

 }

}

dependencies {

 implementation("androidx.core:core-ktx:1.12.0")

 implementation("androidx.lifecycle:lifecycle-runtime-compose:2.7.0")

 implementation("androidx.activity:activity-compose:1.8.2")

 implementation(platform("androidx.compose:compose-bom:2023.10.01"))

 implementation("androidx.compose.ui:ui")

 implementation("com.squareup.okhttp3:okhttp:4.12.0")

 implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.6.2")

}
```

## Data Models

**File:** `Models.kt`@Serializable

```
data class Session(val id: String, val title: String)
```

@Serializable

```
data class Message(
```

```

 val id: String,

 val role: String,

 val content: String,

 val timestamp: Long

)

 @Serializable

 data class StreamEvent(

 val type: String,

 val token: String? = null,

 val toolName: String? = null

)

```

## Network Communication Engine

File: `PiWebClient.kt`

Kotlin

```

class PiWebClient {
 suspend fun createSession() { ... }
 suspend fun abortTask(sessionId: String) { ... }
 suspend fun respondPermission(requestId: String,
 granted: Boolean) { ... }
 fun startStream() { // Includes heartbeat and robust
 event handling }
}

```

```

private val client = OkHttpClient()

```

```

fun startStream(onEvent: (StreamEvent) → Unit): WebSocket {

 val request = Request.Builder().url("$baseUrl/ws/stream").build()

 return client.newWebSocket(request, object : WebSocketListener() {

 override fun onMessage(webSocket: WebSocket, text: String) {

 val event = Json.decodeFromString<StreamEvent>(text)

 onEvent(event)

 }

 })

}

suspend fun executePrompt(prompt: String) {

 // Implementation for POST /api/prompt/execute

}

}

```

## Background Persistence & Notifications

**Service:** `PiKeepAliveService.kt` provides lock-screen persistence.

**Helper:** `NotificationHelper.kt` manages vibrating task alerts for completions and permission requirements.

## State & ViewModel Architecture

File: `MainViewModel.kt`

Kotlin

```

class MainViewModel : ViewModel() {
 val tokenKPI = MutableStateFlow(0)
 val activeToolExecution = MutableStateFlow<ToolCall?>(null)
}

```

```
 val pendingPermissions =
 MutableStateFlow<List<PermissionRequest>>(emptyList())
}
```

```
private val _messages = MutableStateFlow<List<Message>>(emptyList())
```

```
val messages: StateFlow<List<Message>> = _messages.asStateFlow()
```

```
fun sendMessage(text: String) {
```

```
 val newMessage = Message(UUID.randomUUID().toString(), "user", text,
 System.currentTimeMillis())
```

```
 _messages.value += newMessage
```

```
 // Trigger PiWebClient stream here
```

```
}
```

```
}
```

## Material 3 UI Framework

File: Screens.kt

Jetpack Compose screens featuring expandable tool cards, live streaming token progress, session stop button, and permission request dialogs.

# Build & Deployment Instructions

Automated distribution is handled via GitHub Actions to ensure consistent build environments.

1. **Pushing to GitHub:** Ensure all local changes are committed and pushed to the main repository branch.
2. **Compilation:** Upon push, the **Build Android APK** workflow triggers automatically. It configures JDK 17, grants execution rights to the Gradle wrapper, and executes the **assembleDebug** task.
3. **Downloading Artifacts:**
  - Navigate to the **Actions** tab in your GitHub repository.
  - Select the most recent successful workflow run.
  - Scroll to the **Artifacts** section at the bottom of the page.
  - Click **app-debug** to download the compiled APK for testing on physical devices or emulators.

## References

The complete project source and configuration files can be accessed via the following link:

- **Project Archive:**  File
- **Documentation Version:** 1.0.0
- **Last Updated:**  Date